

Trabalho 3 — Processador VLIW

Datapath — VLIW Sagui (pipeline)

Estágio IF

O PC (8 bits) endereça a memória de instruções, que retorna uma palavra de 32 bits — as 4 lanes empacotadas. Um somador calcula PC+1 em paralelo. Instrução e PC+1 são travados no registrador IF/ID.

Registrador IF/ID

Carrega a palavra de 32 bits e PC+1. Em caso de branch tomado no EX, recebe flush e passa a propagar NOP em todos os slots.

Estagio ID

O decodificador recebe a instrução de 32 bits e a divide em 4 túneis de 8 bits — um por lane. Cada túnel passa por dois splitters: o primeiro separa opcode (bits 7-4), ra (bits 3-2) e rb (bits 1-0); o segundo extrai o imediato (bits 3-0). Cada opcode alimenta uma ROM específica da lane que gera os sinais de controle correspondentes.

O banco de registradores possui 8 portas de leitura independentes — Ra e Rb para cada lane — permitindo que todas as lanes leiam operandos simultaneamente sem conflito. As saídas do banco e todos os sinais de controle são travados no registrador ID/EX.

Registrador ID/EX

Composto por 15 registradores de 8 bits (A a O), armazena por lane: operandos Ra e Rb, imediatos, endereços de destino e sinais de controle. Todos os registradores possuem um MUX antes da entrada D com seletor Flush — quando Flush=1 propaga GND, efetivamente inserindo NOP. A organização interna é:

Registrador	Conteúdo
Reg A (8b)	L1_Ra
Reg B (8b)	L1_Rb
Reg C (8b)	L2_Ra
Reg D (8b)	L2_Rb
Reg E (8b)	L3_Ra
Reg F (8b)	L3_Rb
Reg G (8b)	L4_Ra
Reg H (8b)	L4_Rb

Reg I (8b)	PC+1
Reg J (8b)	Imm_L1[3:0] nos bits [7:4] e Imm_L2[3:0] nos bits [3:0]
Reg K (8b)	Imm_L3[3:0] nos bits [7:4] e Imm_L4[3:0] nos bits [3:0]
Reg L (8b)	Rd_L1[1:0] bits[7:6], Rd_L3[1:0] bits[5:4], Rd_L4[1:0] bits[3:2]
Reg M (8b)	MemRead[7], MemWrite[6], L1RegWrite[5], MuxRegIn[4], MuxImm[3], Branch[2], JR[1], Relative[0]
Reg N (8b)	Condition[7], RegWriteL3[6], MUX_ULA_BL3[5], ALUOpL3[4:2], RegWriteL4[1], MUX_ULA_BL4[0]
Reg O (8b)	ALUOpL4[7:5], Sel_Lane1[4:3], bits[2:0] livres

Estágio EX

Lane 1 (LD/ST/MOV)

Recebe Ra e Rb do banco via ID/EX. A saída lane1_out é resultado de um MUX 4:1 controlado por sel_lane1:

sel_lane1	Fonte
00	MemOut da RAM (LD)
01	Imediato montado — MUX entre MOVH e MOVL controlado por mux_imm
10	Rb (MOV)
11	Constante 00

A memória de dados recebe Rb como endereço, Ra como dado de entrada, MemWrite como WE e MemRead como OE. Para ST, escreve Ra no endereço Rb. lane1_out é travado no EX/WB.

Lane 2 (BR/JMP)

Recebe Ra e Rb do banco. Um MUX controlado por condition seleciona entre Ra (para brzr) e R0_out (para brzi) — a saída é splitada bit a bit e cada bit passa por uma porta NOR, gerando o sinal Zero. Este entra em uma porta AND com Branch — o resultado vai para uma porta OR com JR, cuja saída é o Flush. Se Flush=1, o novo PC é selecionado via MUX: R[r] para brzr/jr, PC+1+Imm para brzi.

Lane 3 (ULA A)

Executa operação aritmética ou lógica com o somador Kogge-Stone de 8 bits. Para sub, inverte B e usa carry-in=1. Overflow por wrap around.

Lane 4 (ULA B)

Segunda ULA independente, mesmo Kogge-Stone.

Registrador EX/WB

Composto por 6 registradores de 8 bits (A a F) sem flush. Armazena: lane1_out (Reg A), ula_a (Reg B), ula_b (Reg C), e no Reg D: Rd_L1[7:6], Rd_L3[5:4], Rd_L4[3:2], RegWrite_L1[1], MUX_RegIn_L1[0]. Reg E: RegWrite_L3[7], RegWrite_L4[6].

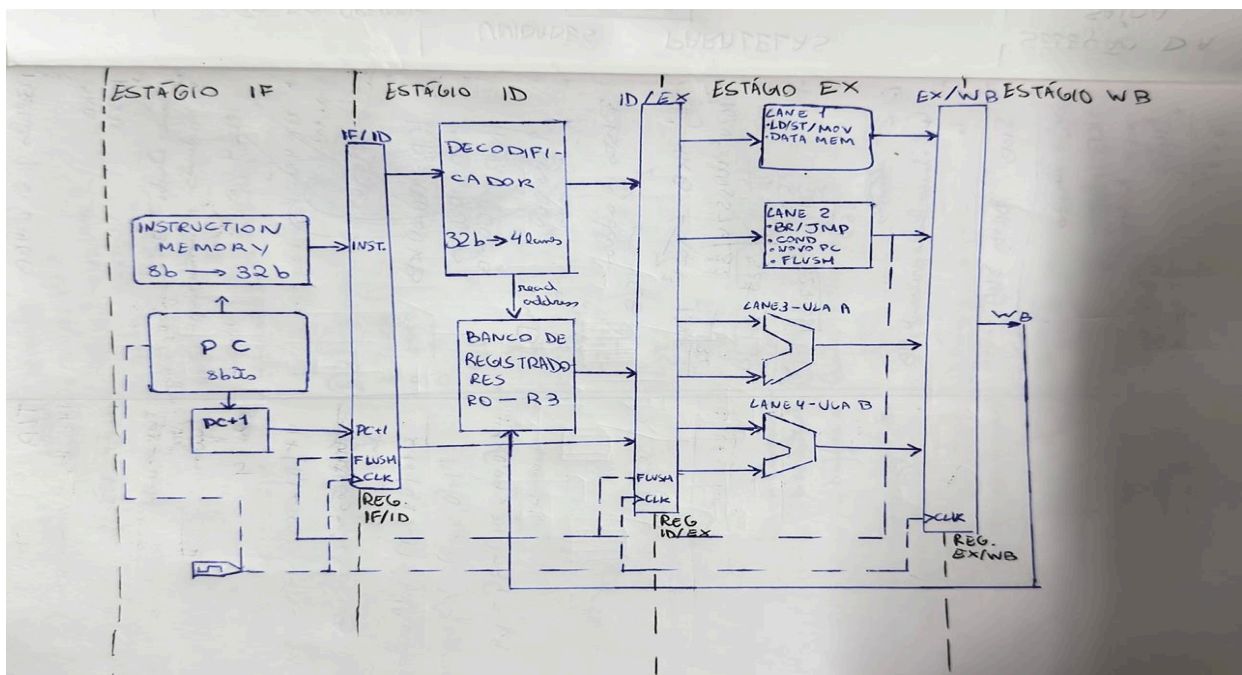
Estágio WB

lane1_out vai direto para WrData1 do banco. Dois MUXes externos selecionam os dados e endereço para a porta de escrita 2:

MUX	Entrada 0	Entrada 1	Seletor	Destino
MUX WrData2	ula_a	ula_b	I3regwrite	WrData2
MUX WrAddr2	rd_l3	rd_l4	I3regwrite	WrAddr2

WE1 é controlado por I1_regwrite. WE2 é a saída de uma porta OR entre I3regwrite e I4regwrite. Lane 2 nunca escreve no banco.

Diagrama de "caixas" - VLIW



Tratamento de Hazards de Dados

O processador VLIW implementa pipeline de 4 estágios (IF, ID, EX, WB) sem forwarding geral entre todas as unidades funcionais. Para garantir a correção da execução, foram adotadas duas estratégias complementares.

A primeira é um forwarding interno à Lane 1, implementado em dois pontos: o resultado do estágio EX/WB da Lane 1 é encaminhado de volta como operando Ra e Rb dentro da própria Lane 1, resolvendo as dependências entre instruções consecutivas do tipo movh→movl e movl→mov. O mesmo forwarding foi estendido à Lane 2, permitindo que brzr e jr usem imediatamente o valor escrito pela instrução anterior da Lane 1 sem necessidade de NOPs.

A segunda estratégia é a inserção estática de NOPs pelo compilador para todas as dependências não cobertas pelo forwarding — especialmente dependências entre a Lane 1 e as lanes ULA (Lanes 3 e 4), onde o resultado de um ld, sub ou add precisa de 3 ciclos para estar disponível no banco de registradores antes de ser lido pela instrução seguinte. Essa abordagem é viável porque o ISA VLIW já pressupõe que o compilador é responsável por garantir a ausência de dependências não resolvidas em hardware.

Diagrama Interno da ULA (Lanes 3 e 4)

Entradas

Sinal	Largura	Descrição
Ra	8 bits	Operando A — sempre do banco de registradores
Rb	8 bits	Operando B — do banco de registradores
Imm	4 bits	Imediato extraído da instrução (para addi)
ALUOp	3 bits	Sinal de controle que seleciona a operação
MUX_ULA_B	1 bit	0=Rb, 1=Imm (com extensão de sinal para 8 bits)

Estágio 1 — Seleção do operando B

Um MUX 2:1 (MUX_ULA_B) seleciona o segundo operando: 0 para Rb (registrador) ou 1 para Imm (imediato com extensão de sinal, para addi). A saída é chamada de B_sel.

Estágio 2 — Unidades paralelas

Kogge-Stone (add/sub/addi)

Recebe Ra e B_sel. Para subtração (ALUOp=001), B_sel é invertido bit a bit antes de entrar no somador e cin=1 e forçado, realizando complemento de 2. Para adição e addi, cin=0. Overflow tratado como wrap around.

Lógica (or/not)

Recebe Ra e B_sel. Realiza OR bit a bit para ALUOp=010 e NOT de B_sel para ALUOp=011 (ignora Ra).

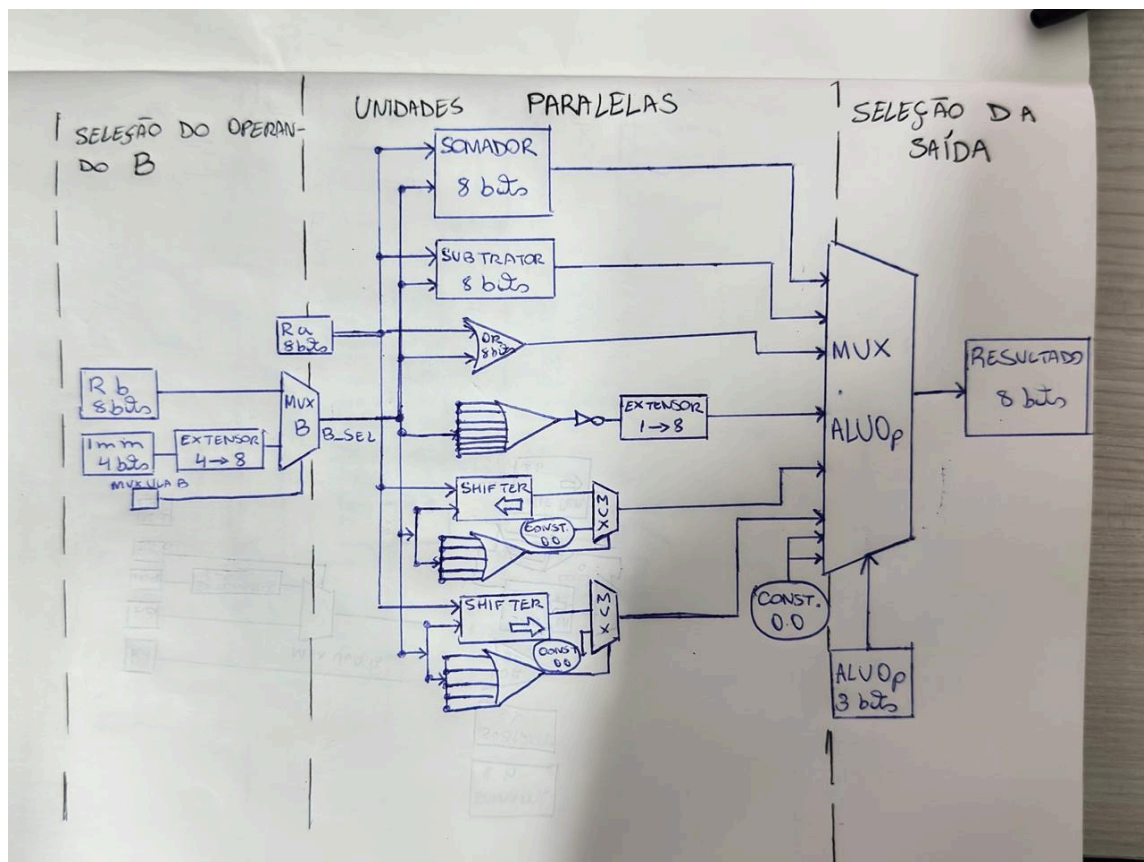
Barrel Shifter (slr/srr)

Recebe Ra como dado e B_sel como quantidade de deslocamento. Realiza shift lógico a esquerda (SLR) ou a direita (SRR) via MUX em cascata.

Estágio 3 — Seleção da saída

ALUOp	Operação	Fonte selecionada
000	add / addi	Kogge-Stone
001	sub	Kogge-Stone (cin=1, B invertido)
010	or	Lógica
011	not	Lógica
100	slr	Barrel Shifter
101	srr	Barrel Shifter
110	-	Don't care
111	nop	RegWrite=0, saída ignorada

Diagrama de "caixas" - ULA



Tabelas de Sinais de Controle

Tabela de Códigos da ULA

ALUOp	Operação
000	add / addi
001	sub
010	or
011	not
100	slr
101	srr
110	-
111	nop

Lane 1 — LD / ST / MOV

Sinais: MemRead, MemWrite, L1RegWrite, sel_lane1 (2 bits), MUX_Imm

Opcode	Mnemônico	MemRead	MemWrite	L1RegWrite	sel_lane1	MUX_Imm
0100	ld	1	0	1	00	-
0101	st	0	1	0	--	-
0110	movh	0	0	1	01	1
0111	movl	0	0	1	01	0
1010	mov	0	0	1	10	-
1111	nop	0	0	0	11	-

Lane 2 — BR / JMP

Sinais: Branch, JR, Relative, Condition

Opcode	Mnemônico	Branch	JR	Relative	Condition
0000	brzr	1	0	0	0 (Ra)
0001	brzi	1	0	1	1 (R0)
0010	jr	0	1	-	-
1111	nop	0	0	-	-

Lanes 3 e 4 — ULA A e ULA B (tabela idêntica)

Sinais: ALUOp (3 bits), RegWrite, MUX_ULA_B

Opcode	Mnemônico	ALUOp	RegWrite	MUX_ULA_B
1000	add	000	1	0
1001	sub	001	1	0
1011	or	010	1	0
1100	not	011	1	0
1101	slr	100	1	0
1110	srr	101	1	0
0011	addi	000	1	1
1111	nop	111	0	-

Tabela Geral de Sinais de Controle

Op co de	Mne môn ico	Lan e	Mem Read	Mem Write	Reg Write	MUX _Regl n	MUX _Imm	Bran ch	JR	Relati ve	Cond ition	ALU Op	MUX_ ULA_ B
00 00	brzr	2	-	-	0	-	-	1	0	0	0	-	-
00 01	brzi	2	-	-	0	-	-	1	0	1	1	-	-
001 0	jr	2	-	-	0	-	-	0	1	-	-	-	-
001 1	addi	3/4	-	-	1	-	-	0	0	-	-	000	1
010 0	ld	1	1	0	1	0	-	0	0	-	-	-	-
010 1	st	1	0	1	0	-	-	0	0	-	-	-	-
011 0	mov h	1	0	0	1	1	1	0	0	-	-	-	-
011 1	movl	1	0	0	1	1	0	0	0	-	-	-	-
100 0	add	3/4	-	-	1	-	-	0	0	-	-	000	0
100 1	sub	3/4	-	-	1	-	-	0	0	-	-	001	0
101 0	mov	1	0	0	1	-	-	0	0	-	-	-	-
101 1	or	3/4	-	-	1	-	-	0	0	-	-	010	0

110 0	not	3/4	-	-	1	-	-	0	0	-	-	011	0
110 1	slr	3/4	-	-	1	-	-	0	0	-	-	100	0
111 0	srr	3/4	-	-	1	-	-	0	0	-	-	101	0
1111	nop	-	0	0	0	-	-	0	0	-	-	-	-

Justificativa para as Instruções Adicionadas

addi ra, imm — Add Immediate

Operação: $R[ra] = R[ra] + Imm$ (signed, 4 bits)

A ausência de uma instrução de soma com imediato no ISA original do Sagui VLIW obrigaria o programador a carregar qualquer constante pequena via movh+movl antes de toda operação aritmética com valor fixo. No programa de teste isso se tornaria especialmente custoso: o loop de soma precisa decrementar o contador e incrementar ponteiros a cada iteração — operações que envolvem constantes pequenas (-1, +1) repetidas 12 vezes. Com o addi, essas operações são expressas diretamente nas lanes ULA sem consumir ciclos extras de inicialização. Além disso, por aceitar imediato com sinal, o addi cobre tanto incrementos quanto decrementos com uma única instrução. No assembly desenvolvido, o addi aparece em todo loop de inicialização e de soma, sendo responsável por reduzir significativamente o número total de palavras VLIW do programa.

mov ra, rb — Move Register

Operação: $R[ra] = R[rb]$

O ISA original do Sagui VLIW não possui instrução de cópia entre registradores. Sem o mov, transferir o conteúdo de um registrador para outro exigiria passar pela memória (st seguido de ld) ou recalculer o valor do zero com movh+movl — ambas as alternativas custosas em ciclos e em uso de lanes. No programa de teste, o mov é usado extensivamente na inicialização para distribuir valores entre registradores após montá-los via movh+movl em R0. Sem ele, cada registrador precisaria de sua própria sequência de dois movh+movl, dobrando o número de instruções de setup. Por ser semanticamente uma operação de transferência e não aritmética, o mov foi alocado na Lane 1 junto com movh e movl, mantendo a coerência do ISA e liberando os opcodes das lanes ULA para operações de fato aritméticas.

Assembly — Soma de Vetores $R = A + B$

Endereços de memória de dados: $A = 0x00$ a $0x0B$, $B = 0x0C$ a $0x17$, $R = 0x18$ a $0x23$

Formato: [Lane1 LD/ST/MOV | Lane2 BR/JMP | Lane3 ULA A | Lane4 ULA B]


```

; --- FASE 1: Inicializa R com zeros ---
; end 0
[ movh R0, 1 | nop | nop | nop ]
; end 1
[ movl R0, 8 | nop | nop | nop ] ; R0=24
; end 2
[ mov R2, R0 | nop | nop | nop ] ; R2=24
; end 3
[ movh R0, 0 | nop | nop | nop ]
; end 4
[ movl R0, 0 | nop | nop | nop ] ; R0=0
; end 5
[ mov R1, R0 | nop | nop | nop ] ; R1=0
; end 6
[ movh R0, 0 | nop | nop | nop ]
; end 7
[ movl R0, 12 | nop | nop | nop ] ; R0=12
; end 8
[ mov R3, R0 | nop | nop | nop ] ; R3=12
; end 9
[ nop | nop | nop | nop ] ; NOP 1
; end 10
[ nop | nop | nop | nop ] ; NOP 2

; init_R = end 11
; end 11
[ st R1, R2 | nop | addi R3, -1 | nop ] ; M[R2]=0, R3--
; end 12
[ nop | nop | addi R2, 1 | nop ] ; R2++
; end 13
[ movh R0, 0 | nop | nop | nop ]
; end 14
[ movl R0, 17 | nop | nop | nop ] ; R0=fim_R=17
; end 15
[ nop | brzr R3, R0 | nop | nop ]
; end 16
[ movh R0, 0 | nop | nop | nop ]
; end 17
[ movl R0, 11 | nop | nop | nop ] ; R0=init_R=11
; end 18
[ nop | jr R0 | nop | nop ]
; fim_R = end 19

; --- FASE 2: Inicializa A com 0,1,...,11 ---
; end 19
[ movh R0, 0 | nop | nop | nop ]
; end 20
[ movl R0, 0 | nop | nop | nop ] ; R0=0
; end 21
[ mov R2, R0 | nop | nop | nop ] ; R2=0
; end 22
[ mov R1, R0 | nop | nop | nop ] ; R1=0
; end 23
[ movh R0, 0 | nop | nop | nop ]
; end 24
[ movl R0, 12 | nop | nop | nop ] ; R0=12
; end 25
[ mov R3, R0 | nop | nop | nop ] ; R3=12
; end 26
[ nop | nop | nop | nop ] ; NOP 1
; end 27
[ nop | nop | nop | nop ] ; NOP 2

; init_A = end 28

```

```

; end 28
[ st R1, R2 | nop | addi R3, -1 | nop ] ; M[R2]=A[i], R3--
; end 29
[ nop | nop | addi R2, 1 | addi R1, 1 ] ; R2++, R1++
; end 30
[ movh R0, 0 | nop | nop | nop ]
; end 31
[ movl R0, 34 | nop | nop | nop ] ; R0=fim_A=34
; end 32
[ nop | brzr R3, R0 | nop | nop ]
; end 33
[ movh R0, 0 | nop | nop | nop ]
; end 34
[ movl R0, 28 | nop | nop | nop ] ; R0=init_A=28
; end 35
[ nop | jr R0 | nop | nop ]
; fim_A = end 36

```

```

; --- FASE 3: Inicializa B com 20,21,...,31 ---
; end 36
[ movh R0, 0 | nop | nop | nop ]
; end 37
[ movl R0, 12 | nop | nop | nop ] ; R0=12
; end 38
[ mov R2, R0 | nop | nop | nop ] ; R2=12
; end 39
[ movh R0, 1 | nop | nop | nop ]
; end 40
[ movl R0, 4 | nop | nop | nop ] ; R0=20
; end 41
[ mov R1, R0 | nop | nop | nop ] ; R1=20
; end 42
[ movh R0, 0 | nop | nop | nop ]
; end 43
[ movl R0, 12 | nop | nop | nop ] ; R0=12
; end 44
[ mov R3, R0 | nop | nop | nop ] ; R3=12
; end 45
[ nop | nop | nop | nop ] ; NOP 1
; end 46
[ nop | nop | nop | nop ] ; NOP 2

```

```

; init_B = end 47
; end 47
[ st R1, R2 | nop | addi R3, -1 | nop ] ; M[R2]=B[i], R3--
; end 48
[ nop | nop | addi R2, 1 | addi R1, 1 ] ; R2++, R1++
; end 49
[ movh R0, 0 | nop | nop | nop ]
; end 50
[ movl R0, 53 | nop | nop | nop ] ; R0=fim_B=53
; end 51
[ nop | brzr R3, R0 | nop | nop ]
; end 52
[ movh R0, 0 | nop | nop | nop ]
; end 53
[ movl R0, 47 | nop | nop | nop ] ; R0=init_B=47
; end 54
[ nop | jr R0 | nop | nop ]
; fim_B = end 55

```

```

; --- FASE 4: Soma R = A + B ---
; end 55
[ movh R0, 0 | nop | nop | nop ]

```

```

; end 56
[ movl R0, 12 | nop | nop | nop ] ; R0=12
; end 57
[ movl R1, R0 | nop | nop | nop ] ; R1=12
; end 58
[ movh R0, 1 | nop | nop | nop ]
; end 59
[ movl R0, 8 | nop | nop | nop ] ; R0=24
; end 60
[ movl R2, R0 | nop | nop | nop ] ; R2=24
; end 61
[ movh R0, 0 | nop | nop | nop ]
; end 62
[ movl R0, 12 | nop | nop | nop ] ; R0=12
; end 63
[ movl R3, R0 | nop | nop | nop ] ; R3=12
; end 64
[ nop | nop | nop | nop ] ; NOP 1
; end 65
[ nop | nop | nop | nop ] ; NOP 2

; soma_vetores = end 66
; end 66
[ movh R0, 0 | nop | nop | nop ]
; end 67
[ movl R0, 12 | nop | nop | nop ]
; end 68
[ nop | nop | nop | nop ] ; NOP 1
; end 69
[ nop | nop | nop | nop ] ; NOP 2
; end 70
[ nop | nop | nop | nop ] ; NOP 3
; end 71
[ nop | nop | sub R0, R3 | nop ] ; R0=12-R3=&A[i]
; end 72
[ nop | nop | nop | nop ] ; NOP 1
; end 73
[ nop | nop | nop | nop ] ; NOP 2
; end 74
[ nop | nop | nop | nop ] ; NOP 3
; end 75
[ ld R0, R0 | nop | nop | nop ] ; R0=A[i]
; end 76
[ nop | nop | nop | nop ] ; NOP 1
; end 77
[ nop | nop | nop | nop ] ; NOP 2
; end 78
[ nop | nop | nop | nop ] ; NOP 3
; end 79
[ ld R1, R1 | nop | nop | nop ] ; R1=B[i]
; end 80
[ nop | nop | nop | nop ] ; NOP 1
; end 81
[ nop | nop | nop | nop ] ; NOP 2
; end 82
[ nop | nop | nop | nop ] ; NOP 3
; end 83
[ nop | nop | add R0, R1 | nop ] ; R0=A[i]+B[i]
; end 84
[ nop | nop | nop | nop ] ; NOP 1
; end 85
[ nop | nop | nop | nop ] ; NOP 2
; end 86
[ nop | nop | nop | nop ] ; NOP 3

```

```

; end 87
[ st R0, R2 | nop | nop | nop ] ; M[R2]=resultado
; end 88
[ nop | nop | addi R1, 1 | addi R2, 1 ] ; B++, R++
; end 89
[ nop | nop | addi R3, -1 | nop ] ; R3--
; end 90
[ movh R0, 6 | nop | nop | nop ]
; end 91
[ movl R0, 0 | nop | nop | nop ] ; R0=0x60=96=fim
; end 92
[ nop | brzr R3, R0 | nop | nop ]
; end 93
[ movh R0, 4 | nop | nop | nop ]
; end 94
[ movl R0, 2 | nop | nop | nop ] ; R0=0x42=66=soma_vetores
; end 95
[ nop | jr R0 | nop | nop ]
; fim = end 96
[ nop | nop | nop | nop ]

```

Hex da Memória de Instruções

Endereçamento por palavra (32 bits por endereço). Formato por instrução: [Lane1][Lane2][Lane3][Lane4].

End.	Lane1	Lane2	Lane3	Lane4	Instrução
0	61	FF	FF	FF	movh R0, 1
1	78	FF	FF	FF	movl R0, 8
2	A8	FF	FF	FF	mov R2, R0
3	60	FF	FF	FF	movh R0, 0
4	70	FF	FF	FF	movl R0, 0
5	A4	FF	FF	FF	mov R1, R0
6	60	FF	FF	FF	movh R0, 0
7	7C	FF	FF	FF	movl R0, 12
8	AC	FF	FF	FF	mov R3, R0
9	56	FF	3F	FF	st R1,R2 addi R3,-1
10	FF	FF	39	FF	addi R2, 1
11	60	FF	FF	FF	movh R0, 0
12	7E	FF	FF	FF	movl R0, 14
13	FF	0C	FF	FF	brzr R3, R0
14	FF	1A	FF	FF	brzi -6
15	60	FF	FF	FF	movh R0, 0
16	70	FF	FF	FF	movl R0, 0

17	A8	FF	FF	FF	mov R2, R0
18	A4	FF	FF	FF	mov R1, R0
19	60	FF	FF	FF	movh R0, 0
20	7C	FF	FF	FF	movl R0, 12
21	AC	FF	FF	FF	mov R3, R0
22	56	FF	3F	FF	st R1,R2 addi R3,-1
23	FF	FF	39	35	addi R2,1 addi R1,1
24	60	FF	FF	FF	movh R0, 0
25	7B	FF	FF	FF	movl R0, 27
26	FF	0C	FF	FF	brzr R3, R0
27	FF	1A	FF	FF	brzi -6
28	60	FF	FF	FF	movh R0, 0
29	7C	FF	FF	FF	movl R0, 12
30	A8	FF	FF	FF	mov R2, R0
31	61	FF	FF	FF	movh R0, 1
32	74	FF	FF	FF	movl R0, 4
33	A4	FF	FF	FF	mov R1, R0
34	60	FF	FF	FF	movh R0, 0
35	7C	FF	FF	FF	movl R0, 12
36	AC	FF	FF	FF	mov R3, R0
37	56	FF	3F	FF	st R1,R2 addi R3,-1
38	FF	FF	39	35	addi R2,1 addi R1,1
39	60	FF	FF	FF	movh R0, 0
40	7A	FF	FF	FF	movl R0, 42
41	FF	0C	FF	FF	brzr R3, R0
42	FF	1A	FF	FF	brzi -6
43	60	FF	FF	FF	movh R0, 0
44	7C	FF	FF	FF	movl R0, 12
45	A4	FF	FF	FF	mov R1, R0
46	61	FF	FF	FF	movh R0, 1
47	78	FF	FF	FF	movl R0, 8
48	A8	FF	FF	FF	mov R2, R0
49	60	FF	FF	FF	movh R0, 0
50	7C	FF	FF	FF	movl R0, 12

51	AC	FF	FF	FF	mov R3, R0
52	60	FF	FF	FF	movh R0, 0 (soma_vetores)
53	7C	FF	FF	FF	movl R0, 12
54	FF	FF	93	FF	sub R0, R3
55	40	FF	FF	FF	ld R0, R0
56	45	FF	FF	FF	ld R1, R1
57	FF	FF	81	FF	add R0, R1
58	52	FF	FF	FF	st R0, R2
59	FF	FF	35	39	addi R1,1 addi R2,1
60	FF	FF	3F	FF	addi R3, -1
61	64	FF	FF	FF	movh R0, 4
62	73	FF	FF	FF	movl R0, 3
63	FF	0C	FF	FF	brzr R3, R0
64	63	FF	FF	FF	movh R0, 3
65	74	FF	FF	FF	movl R0, 4
66	FF	20	FF	FF	jr R0
67	FF	FF	FF	FF	nop (fim)

Conteúdo da memória de instruções:

v3.0 hex words addressed

```

000: 61ffffff 78ffffff a8ffffff 60ffffff 70ffffff a4ffffff 60ffffff 7cffffff
008: acffffff ffffffff ffffffff 56ff3fff ffff39ff 60ffffff 71ffffff ff0cfff
010: 60ffffff 7bffffff ff20fff 60ffffff 70ffffff a8ffffff a4ffffff 60ffffff
018: 7cffffff acffffff ffffffff ffffffff 56ff3fff ffff3935 60ffffff 72ffffff
020: ff0cfff 60ffffff 7cffffff ff20fff 60ffffff 7cffffff a8ffffff 61ffffff
028: 74ffffff a4ffffff 60ffffff 7cffffff acffffff ffffffff ffffffff 56ff3fff
030: ffff3935 60ffffff 75ffffff ff0cfff 60ffffff 7ffffff ff20fff 60ffffff
038: 7cffffff a4ffffff 61ffffff 78ffffff a8ffffff 60ffffff 7cffffff acffffff
040: ffffffff ffffffff 60ffffff 7cffffff ffffffff ffffffff ffffffff ffff93ff
048: ffffffff ffffffff ffffffff 40ffffff ffffffff ffffffff ffffffff 45ffffff
050: ffffffff ffffffff ffffffff ffff81ff ffffffff ffffffff ffffffff 52ffffff
058: ffff3539 ffff3fff 66ffffff 70ffffff ff0cfff 64ffffff 72ffffff ff20fff
060: ffffffff

```